



CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)



Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Assignment-1

Richardson Maturity Model (Level 0 - 3)

Problem Definition: -

- Evaluate the Task Management API built in Practical 4 against the Richardson Maturity Model.
- Identify which level (0, 1, 2, or 3) the current API satisfies and justify your answer.
- Improve the API to reach at least Level 2 if it does not already satisfy it.
- Document the evaluation and improvement in a MATURITY.md file in your assignment repository.

Evaluation Conclusion & Level 2 Justification:

The Task Management API built in Practical 4 satisfies Level 2 of the Richardson Maturity Model based on the following criteria:

- 1. HTTP Verbs Mapping: GET /tasks retrieves the task list safely. GET /tasks/:id fetches a specific task. POST /tasks creates a new task. PUT /tasks/:id updates an existing task, and DELETE /tasks/:id removes a task.
- . Meaningful Status Codes: Returns 200 OK for successful GET/PUT/DELETE, 201 Created for POST, 400 Bad Request for validation failures, 404 Not Found for missing tasks/routes, and 500 Internal Server Error for server failures without leaking stack traces.

Richardson Maturity Model(RMM) Evaluation Table:

Level	Criterion	Satisfied?	Evidence & Justification
Level 0	The Swamp of POX (Single URI & HTTP POST)	Yes (Passed & Exceeded)	The API does not use a single monolithic endpoint like /api. It uses individual resource URIs (/tasks, /tasks/:id).
Level 1	Resources (Separate URIs for entities)	Yes	Tasks are exposed as discrete resource URIs (/tasks for collection, /tasks/:id for specific task resources).
Level 2	HTTP Verbs & Status Codes	Yes	Standard HTTP verbs are strictly mapped (GET, POST, PUT, DELETE) and meaningful status codes are returned (200, 201, 400, 404, 500).
Level 3	HATEOAS (Hypermedia Controls)	No (Targeted for awareness)	Response payloads currently return raw JSON resource data without hypermedia _links navigation objects.

Level 3: HATEOAS Awareness & Representation:

To achieve Level 3 maturity, response bodies must include a hypermedia links section (`_links`) allowing clients to dynamically discover available actions:

```
{
  "id": "123",
  "title": "Task A",
  "completed": false,
  "_links": {
    "self": { "href": "/tasks/123", "method": "GET" },
    "update": { "href": "/tasks/123", "method": "PUT" },
    "delete": { "href": "/tasks/123", "method": "DELETE" },
    "collection": { "href": "/tasks", "method": "GET" }
  }
}
```

Analysis: Why Most Production APIs Stop at Level 2:

- **Payload Overhead & Bandwidth:** Adding `_links` objects to every item in large JSON arrays increases payload size, consuming additional bandwidth.
- **Client Architecture:** Front-end applications (React, Angular, Mobile Apps) hardcode client-side API calls. Dynamic link parsing adds code complexity with minimal gain.
- **Documentation Standards:** OpenAPI / Swagger specifications provide clear, versioned documentation out of the box, making dynamic link discovery redundant.
- **Lack of Standardization:** Competing formats (HAL, JSON-LD, Siren) prevent a single standard, leading teams to stick to clean Level 2 JSON.

Github Link:- [krunal9116/assignment-w4-D25IT137](https://github.com/krunal9116/assignment-w4-D25IT137)